# Angular Signals Intro by Nickk

Angular Signals were introduced as a **developer preview** in Angular 16 (released in May 2023). They were later promoted to **stable** APIs in Angular 17 and Angular 20 for the core primitives.

---

Angular enters a new era of reactivity with the introduction of **Signals**
Their core primitives are `signal()`, `computed()`, and `effect()`.

Angular's new reactive system is built on three essential primitives: `signal()`, `computed()`, and `effect()`. These primitives create a direct and predictable relationship between application state and the user interface, streamlining how changes flow through an Angular application.

## signal()

A `signal()` is the starting point. It encapsulates a single piece of state, providing both a getter and setter for its value.
When a signal's value is updated, Angular automatically tracks which parts of the application have read from it and triggers updates precisely where needed.

- This behavior removes the need for explicit subscriptions, manual event listeners, or redundant change detection triggers.
- A signal can be thought of as a reactive variable, but one that carries with it a deep understanding of its consumers across the application.

## computed()

Built on top of signals, represent derived data.
A `computed()` function automatically recalculates itself whenever any of the signals it depends on change.
Declarative way to express relationships between different pieces of state.
*This encourages a design where derived data is defined once and reused consistently, without risk of stale values or unintended inconsistencies. Computed values are lazy by design, updating only when actively used, which leads to highly efficient UI updates.*

## effect()

The final primitive, `effect()`, introduces controlled side effects into the reactive system.
An `effect()` function runs whenever one of its dependent signals changes, executing logic that falls outside pure state transformations. This is where Angular developers can interact with services, perform DOM manipulations, or trigger external processes in a safe, signal-driven manner. Effects provide a clean boundary between pure reactivity and the inevitable imperative code that all real-world applications require. Effects must be used inside an injection context, such as the constructor.

Signal

It is a declaration of a dynamic value.

It is a reactive primitive, ng16+

Represents a value that ng can track

Ng knows who depends on it and updates only those parts

value + dependency tracking engine

1. Signal (signal<T>)

A signal is a wrapper around a value that can notify interested consumers when that value changes.

Think of it like a "reactive variable" - when the value changes, anything using it automatically knows about it.

The whole point of Angular signals: to allow the Angular framework to optimize change detection updates.

```
const count = signal(0);
    -> count is not number 0
    -> nor it is a plain object
    -> it is function object.

calling count() -> returns the current value.

this fn object also has methods, .set(), .update(fn), etc..
```

Angular Signals are a modern reactive primitive used for managing and tracking state changes within an application, allowing the framework to optimize rendering updates. They provide a way for data to notify interested consumers when its value changes, leading to more performant and straightforward reactive code.

computed(): Defines a read-only signal whose value is derived from other signals. It automatically recalculates when any of its dependent signals change and the value is accessed. computed signals are lazily evaluated and memoized for performance.

```
firstName = signal('Daniel');
```

The computed is a way to combine multiple signals into a single read-only signal.

```
firstName = signal('Daniel');
lastName = signal('Kreider');
fullName = computed(() => firstName() + ' ' + lastName());
```

And last of all, the effect is an operation that runs when a signal value changes. This allows you to programmatically react to signal changes.

```
firstName = signal('Daniel');
lastName = signal('Kreider');
fullName = computed(() => firstName() + ' ' + lastName());

effect(() => {
  console.log(`The current name is: ${fullName()}`);
});
```

> Signals are primitives, so we can use them outside the component class

## Using Signals for State Management

Signals can manage various scopes of state, from simple component-level UI toggles to complex, application-wide data.

## Component-Level State

For simple, local state, signals can be defined directly within the component:

```
import { Component, signal } from '@angular/core';

@Component({
  selector: 'app-counter',
  template: `
    <div>
      <p>Count: {{ count() }}</p>
      <button (click)="increment()">Increment</button>
    </div>
  `,
})
export class CounterComponent {
  count = signal(0); //

  increment() {
    this.count.update(value => value + 1); //
  }
}
```

The template automatically re-renders with the minimal necessary changes when `count` is updated, without using the `async` pipe or manual subscriptions.

## Shared State in Services

For state that needs to be shared across multiple components, signals can be encapsulated within an injectable service:

```
import { Injectable, signal, computed } from '@angular/core';

@Injectable({ providedIn: 'root' })
export class CartService {
  private _cartItems = signal<string[]>([]); //

  readonly cartCount = computed(() => this._cartItems().length); // Derived state

  addItem(item: string) {
    this._cartItems.update(items => [...items, item]); //
  }

  // Expose read-only signals for components to consume
  get cartItems() {
    return this._cartItems.asReadonly();
  }
}
```

Components can then inject the service and read the `cartItems` and `cartCount` signals:

```
import { Component, inject } from '@angular/core';
import { CartService } from './cart.service';

@Component({ ... })
export class CartIconComponent {
  // Inject and read signals directly in the template
  readonly cartService = inject(CartService);
}
```